AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

KANTAMRAJU SREEJA

CH.SC.U4CSE24255

WORKSHEET-1

WORKSHEET- 1

EX.NO : 1

DATE : 09-10-26

TITLE : Implementation and Verification of Single-Qubit, Two-Qubit and Three-Qubit Quantum Gates and Circuits Using Qiskit

AIM:

To construct and simulate quantum circuits using single-qubit, two-qubit and three-qubit gates in Qiskit, to verify each gate against its matrix and truth table, and to prepare the entangled Bell and GHZ states.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

A qubit is the basic unit of quantum information. Its state is written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1$$

where α and β are complex numbers called amplitudes. A measurement gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. A **quantum gate** changes the state of one or more qubits. A gate acting on n qubits is a $2^n \times 2^n$ unitary matrix U, which means $U^\dagger U = I$. This keeps the total probability equal to 1 and makes every gate reversible.

Single-qubit gates. The Pauli-X gate swaps $|0\rangle$ and $|1\rangle$, so it is the quantum NOT gate. The Pauli-Z gate leaves $|0\rangle$ unchanged and changes the sign of $|1\rangle$. The Pauli-Y gate does both, with a factor of i. The S and T gates add phases of $\pi/2$ and $\pi/4$ to $|1\rangle$. The Hadamard gate creates an equal superposition:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Two-qubit gates. A controlled gate acts on a target qubit only when the control qubit is $|1\rangle$. The CNOT (controlled-X) gate maps $|a, b\rangle$ to $|a, a \oplus b\rangle$, where a is the control, b is the target and $\oplus$ is addition modulo 2 (XOR). With the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ and the control written first, its matrix is

$$\mathrm{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The CZ gate multiplies $|11\rangle$ by -1 and leaves the other three basis states unchanged. The SWAP gate exchanges the states of two qubits. A Hadamard gate on one qubit followed by a CNOT produces a Bell state, in which the two qubits are **entangled**: neither qubit has a state of its own, and their measurement results always agree.

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Three-qubit gates. The Toffoli gate (CCX, controlled-controlled-NOT) flips the target only when both controls are 1: $|a, b, c\rangle \to |a, b, c \oplus ab\rangle$. With the target starting at 0 it computes the AND of the two controls, so any classical logic circuit can be built reversibly from Toffoli gates. The Fredkin gate (CSWAP, controlled-SWAP) swaps two target qubits only when the control is 1. Each is an 8×8 matrix that rearranges the basis states. A Hadamard gate followed by two CNOT gates gives the three-qubit GHZ state:

$$|\mathrm{GHZ}\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

Quantum hardware offers only one-qubit and two-qubit gates, so a three-qubit gate is decomposed before it is run. The standard decomposition of the Toffoli gate uses six CNOT gates.

Bit ordering in Qiskit. Qiskit numbers qubits from the right: qubit 0 is the rightmost bit of a printed result. A two-qubit circuit with X on qubit 0 prints `01`. Most textbooks write the first qubit on the left, so the matrix Qiskit prints for `cx(0, 1)` has its rows in a different order from the matrix above, although it is the same gate. State which ordering is used whenever a hand calculation is compared with Qiskit output.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit.__version__`.

2. Open Jupyter Notebook or Visual Studio Code.

3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and

`Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`.

4. Create a `QuantumCircuit` with the required number of qubits (one, two or three) and, where a measurement is needed, the same number of classical bits.

5. Prepare the required input state with X gates, then apply the gate under study (`x`, `y`, `z`, `h`, `s`, `t`, `cx`, `cz`, `swap`, `ccx`, `cswap`).

6. Draw the circuit with `qc.draw("mpl")` before running it.

7. Display the exact state with `Statevector(qc)` and the gate matrix with `Operator(qc)`. Do this before any measurement is added.

8. Add measurements and run the circuit on `AerSimulator` with 1024 shots: `sim.run(transpile(qc, sim), shots=1024).result().get_counts()`. Plot the counts with `plot_histogram`.

9. Repeat for every input basis state and fill in the truth table of the gate.

10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

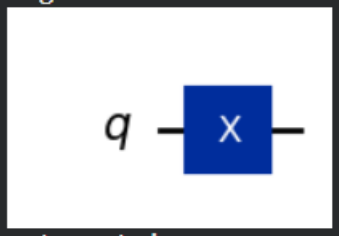
Program 1: Apply the X, Y, Z, H, S and T gates, one at a time, to $|0\rangle$ and to $|1\rangle$. Display the resulting statevector and the gate matrix for each, and compare them with the matrices in the theory.

OUTPUT:

```python
gates = {
    "X": lambda qc, q: qc.x(q),
    "Y": lambda qc, q: qc.y(q),
    "Z": lambda qc, q: qc.z(q),
    "H": lambda qc, q: qc.h(q),
    "S": lambda qc, q: qc.s(q),
    "T": lambda qc, q: qc.t(q),
}

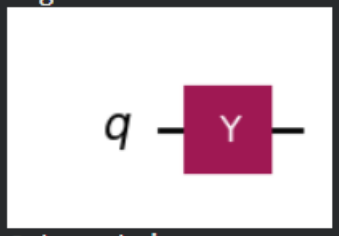
for gate_name, apply_gate in gates.items():
    qc_gate = QuantumCircuit(1)
    apply_gate(qc_gate, 0)
    print(f"\n{gate_name} gate circuit:")
    display(qc_gate.draw("mpl"))
    print("Gate matrix:")
    display(Operator(qc_gate).data)
    for input_bit in [0, 1]:
        qc = QuantumCircuit(1)
        if input_bit == 1:
            qc.x(0)
        apply_gate(qc, 0)
        print(f"{gate_name} applied to |{input_bit}>:", Statevector.from_instruction(qc).data)
```

```
X gate circuit:
```



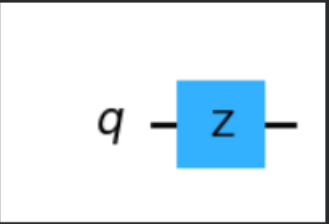

```
Gate matrix:
array([[0.+0.j, 1.+0.j],
       [1.+0.j, 0.+0.j]])
X applied to |0>: [0.+0.j 1.+0.j]
X applied to |1>: [1.+0.j 0.+0.j]

Y gate circuit:
```



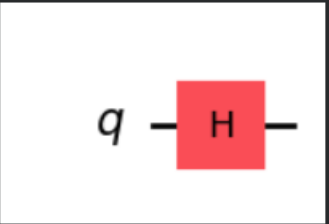

```
Gate matrix:
array([[0.+0.j, 0.-1.j],
       [0.+1.j, 0.+0.j]])
Y applied to |0>: [0.+0.j 0.+1.j]
Y applied to |1>: [0.-1.j 0.+0.j]
```

```
Z gate circuit:
```



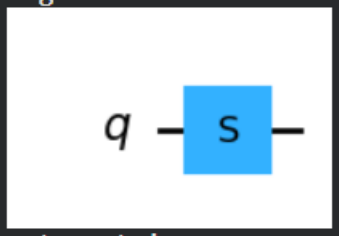

```
Gate matrix:
array([[ 1.+0.j,  0.+0.j],
       [ 0.+0.j, -1.+0.j]])
Z applied to |0>: [1.+0.j 0.+0.j]
Z applied to |1>: [ 0.+0.j -1.+0.j]

H gate circuit:
```



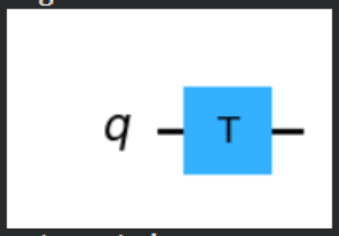

```
Gate matrix:
array([[ 0.70710678+0.j,  0.70710678+0.j],
       [ 0.70710678+0.j, -0.70710678+0.j]])
H applied to |0>: [0.70710678+0.j 0.70710678+0.j]
H applied to |1>: [ 0.70710678+0.j -0.70710678+0.j]
```

```
S gate circuit:
```




```
Gate matrix:
array([[1.+0.j, 0.+0.j],
       [0.+0.j, 0.+1.j]])
S applied to |0>: [1.+0.j 0.+0.j]
S applied to |1>: [0.+0.j 0.+1.j]

T gate circuit:
```

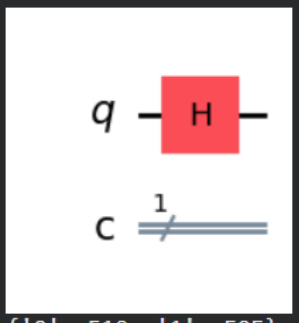



```
Gate matrix:
array([[1.         +0.j         , 0.         +0.j         ],
       [0.         +0.j         , 0.70710678+0.70710678j]])
T applied to |0>: [1.+0.j 0.+0.j]
T applied to |1>: [0.         +0.j         0.70710678+0.70710678j]
```

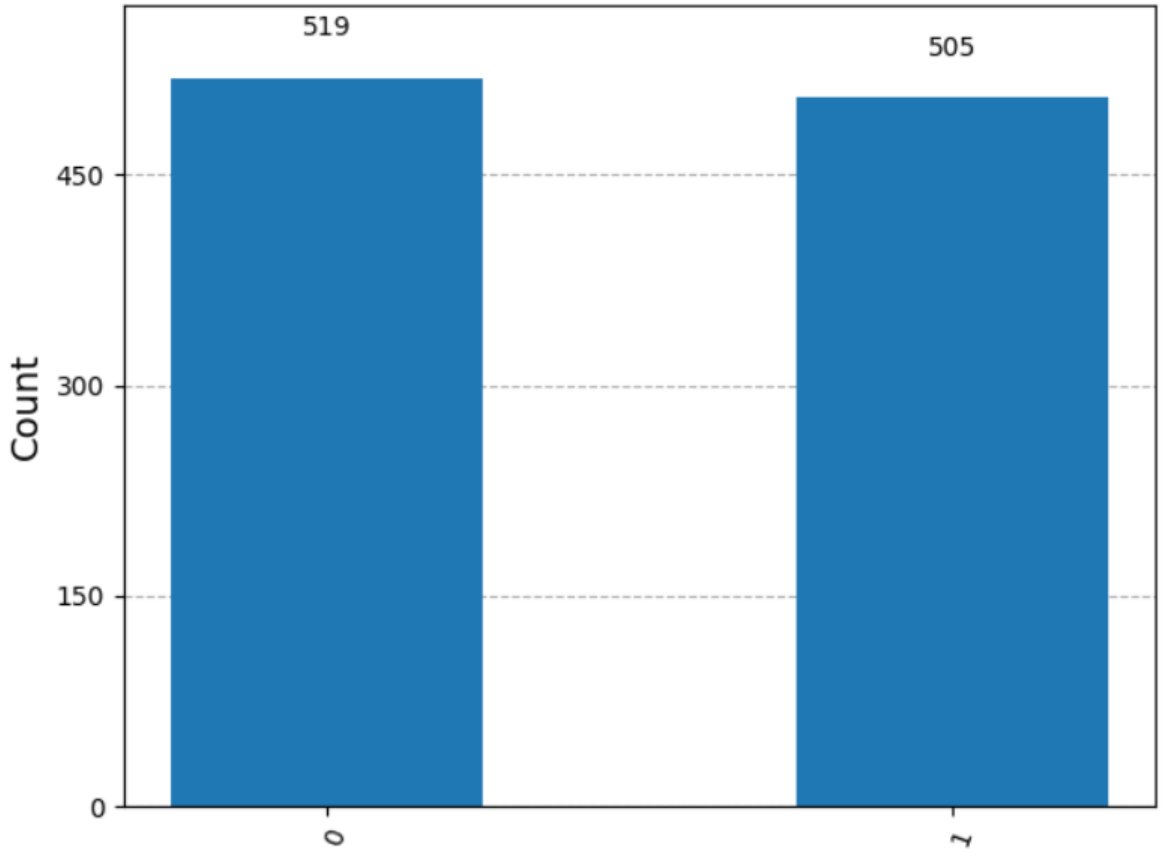
Program 2: Create a single-qubit circuit with a Hadamard gate, measure it with 1024 shots on the Aer simulator and plot the histogram. Explain why the two counts are close to, but not exactly, 512.

OUTPUT:

```
qc = QuantumCircuit(1, 1)
qc.h(0)
display(qc.draw("mpl"))
measured = qc.copy()
measured.measure(0, 0)
counts = sim.run(transpile(measured, sim), shots=SHOTS).result().get_counts()
print(counts)
display(plot_histogram(counts))
```




```
{'0': 519, '1': 505}
```

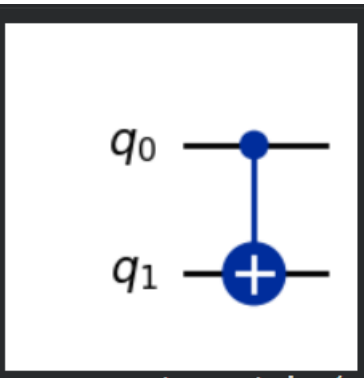


Program 3: Implement the CNOT gate and verify its truth table for all four input states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Display its matrix with `Operator` and explain how it relates to the textbook matrix.

OUTPUT:

```
qc = QuantumCircuit(2)
qc.cx(0, 1)
display(qc.draw("mpl"))
print("CNOT operator matrix (Qiskit ordering):")
display(Operator(qc).data)

print("Input q1q0 -> output q1q0")
for q0 in [0, 1]:
    for q1 in [0, 1]:
        test = QuantumCircuit(2)
        if q0: test.x(0)
        if q1: test.x(1)
        test.cx(0, 1)
        sv = Statevector.from_instruction(test)
        index = int(abs(sv.data).argmax())
        print(f"{q1}{q0} -> {format(index, '02b')}")
```




```
CNOT operator matrix (Qiskit ordering):
array([[1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j],
       [0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j]])
Input q1q0 -> output q1q0
00 -> 00
10 -> 10
01 -> 11
11 -> 01
```

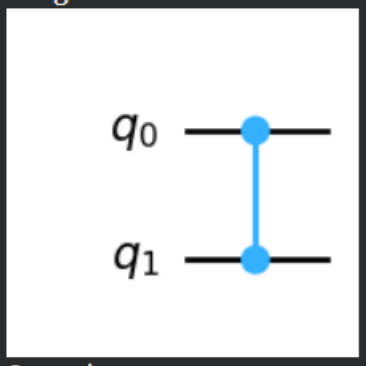
Program 4: Implement the CZ and SWAP gates and verify each on all four basis states. Show, by comparing operators, that a CNOT with a Hadamard gate on the target before and after it is equal to a CZ gate.

OUTPUT:

```
for name in ["CZ", "SWAP"]:
    print(f"\n{name} gate")
    gate = QuantumCircuit(2)
    if name == "CZ":
        gate.cz(0, 1)
    else:
        gate.swap(0, 1)
    display(gate.draw("mpl"))
    print("Operator:")
    display(Operator(gate).data)
    print("Input q1q0 -> output statevector")
    for q1 in [0, 1]:
        for q0 in [0, 1]:
            test = QuantumCircuit(2)
            if q0: test.x(0)
            if q1: test.x(1)
            if name == "CZ": test.cz(0, 1)
            else: test.swap(0, 1)
            print(f"{q1}{q0} -> {Statevector.from_instruction(test).data}")

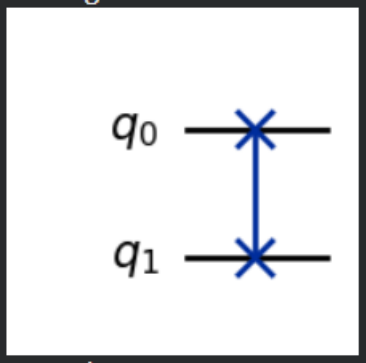
circuit_identity = QuantumCircuit(2)
circuit_identity.h(1)
circuit_identity.cx(0, 1)
circuit_identity.h(1)
cz_circuit = QuantumCircuit(2)
cz_circuit.cz(0, 1)
print("\nH(target) - CNOT - H(target) equivalent to CZ:",
      Operator(circuit_identity).equiv(Operator(cz_circuit)))
```

```
CZ gate
```




```
Operator:
array([[ 1.+0.j,  0.+0.j,  0.+0.j,  0.+0.j],
       [ 0.+0.j,  1.+0.j,  0.+0.j,  0.+0.j],
       [ 0.+0.j,  0.+0.j,  1.+0.j,  0.+0.j],
       [ 0.+0.j,  0.+0.j,  0.+0.j, -1.+0.j]])
Input q1q0 -> output statevector
00 -> [1.+0.j 0.+0.j 0.+0.j 0.+0.j]
01 -> [0.+0.j 1.+0.j 0.+0.j 0.+0.j]
10 -> [0.+0.j 0.+0.j 1.+0.j 0.+0.j]
11 -> [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]
```

```
SWAP gate
```




```
Operator:
array([[1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j]])
Input q1q0 -> output statevector
00 -> [1.+0.j 0.+0.j 0.+0.j 0.+0.j]
01 -> [0.+0.j 0.+0.j 1.+0.j 0.+0.j]
10 -> [0.+0.j 1.+0.j 0.+0.j 0.+0.j]
11 -> [0.+0.j 0.+0.j 0.+0.j 1.+0.j]

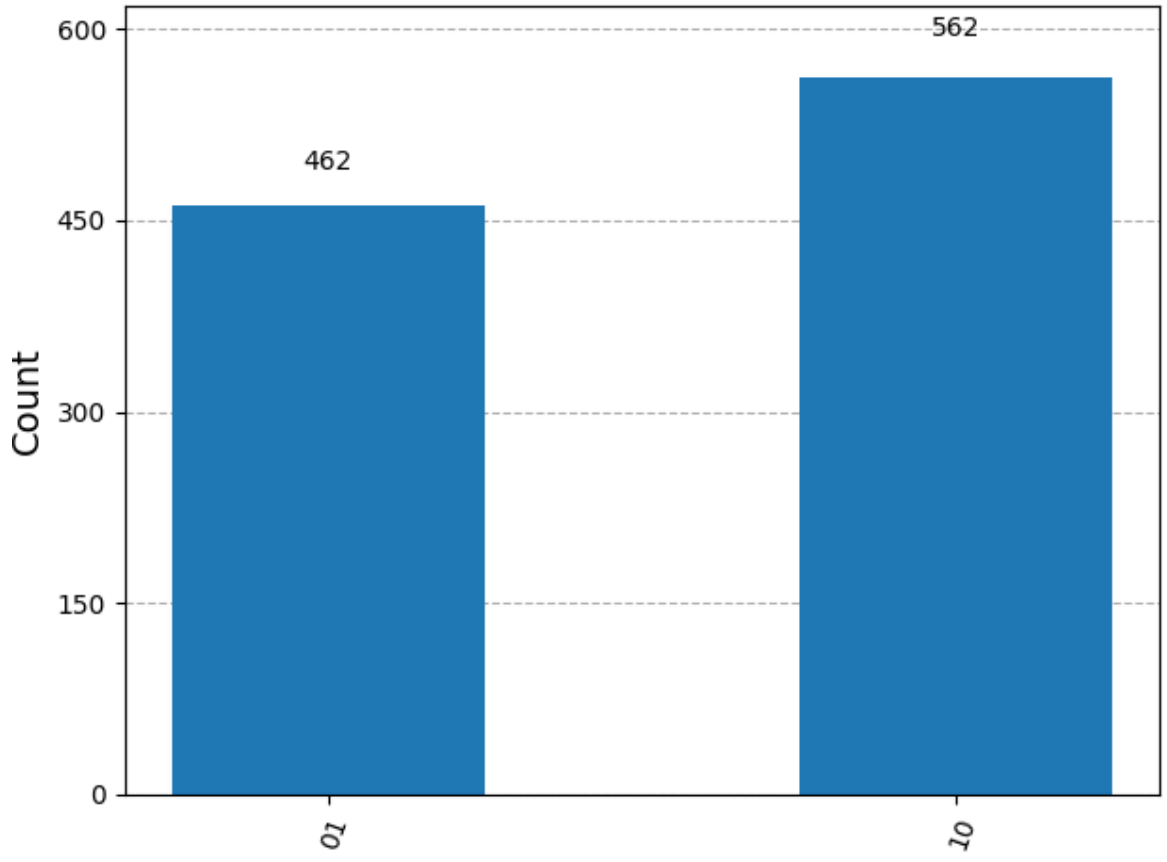
H(target) – CNOT – H(target) equivalent to CZ: True
```

Program 5: Prepare the four Bell states $|\Phi^+\rangle, |\Phi^-\rangle, |\Psi^+\rangle, |\Psi^-\rangle$. Display the statevector and the measurement histogram of each.

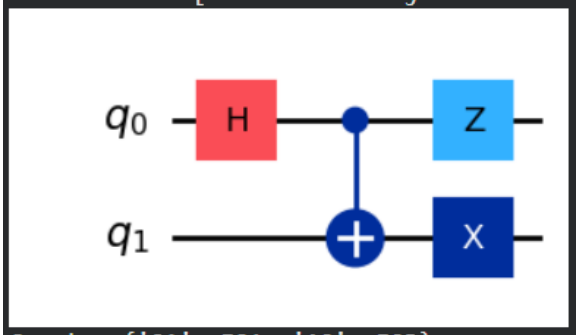
OUTPUT:

```
bell_preparations = {
    "Phi+": [],
    "Phi-": [("z", 0)],
    "Psi+": [("x", 1)],
    "Psi-": [("x", 1), ("z", 0)],
}

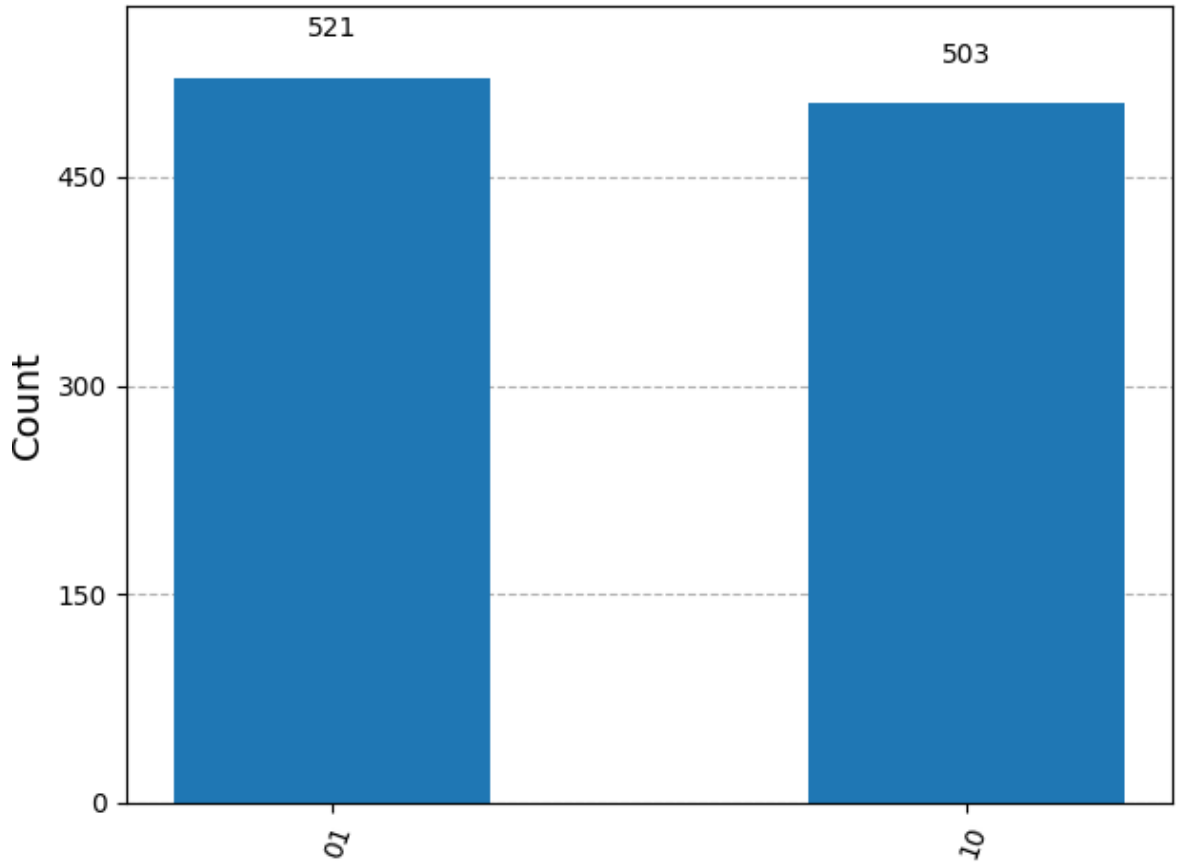
for name, operations in bell_preparations.items():
    qc = QuantumCircuit(2)
    qc.h(0)
    qc.cx(0, 1)
    for op, qubit in operations:
        getattr(qc, op)(qubit)
    print(f"\nBell state {name}")
    print("Statevector:", Statevector.from_instruction(qc).data)
    display(qc.draw("mpl"))
    measured = qc.copy()
    measured.measure_all()
    counts = sim.run(transpile(measured, sim), shots=SHOTS).result().get_counts()
    print("Counts:", counts)
    display(plot_histogram(counts))
```



```
Bell state Psi-
Statevector: [ 0.         +0.j -0.70710678+0.j  0.70710678+0.j  0.         +0.j]
```




```
Counts: {'01': 521, '10': 503}
```

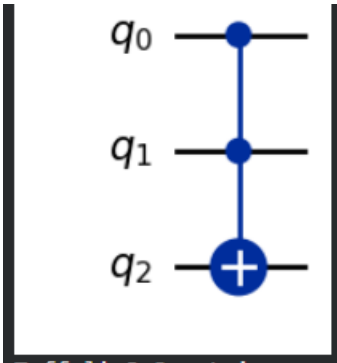


Program 6: Implement the Toffoli (CCX) gate. Verify its truth table for all eight input states and display its 8 × 8 matrix.

OUTPUT:

```
qc = QuantumCircuit(3)
qc.ccx(0, 1, 2)
display(qc.draw("mpl"))
print("Toffoli 8x8 matrix:")
display(Operator(qc).data)

print("Input q2q1q0 -> output q2q1q0")
for q2 in [0, 1]:
    for q1 in [0, 1]:
        for q0 in [0, 1]:
            test = QuantumCircuit(3)
            if q0: test.x(0)
            if q1: test.x(1)
            if q2: test.x(2)
            test.ccx(0, 1, 2)
            sv = Statevector.from_instruction(test)
            index = int(abs(sv.data).argmax())
            print(f"{q2}{q1}{q0} -> {format(index, '03b')}")
```

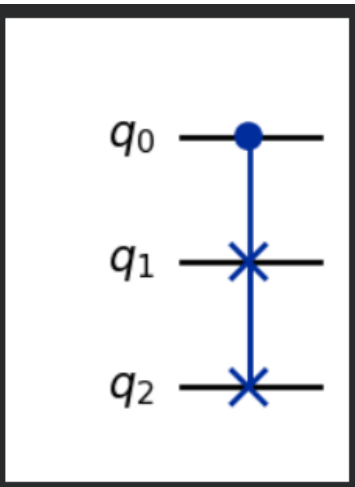



```
Toffoli 8x8 matrix:
array([[1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j]])
Input q2q1q0 -> output q2q1q0
000 -> 000
001 -> 001
010 -> 010
011 -> 111
100 -> 100
101 -> 101
110 -> 110
111 -> 011
```

Program 7: Implement the Fredkin (CSWAP) gate. Verify its truth table for all eight input states and state which inputs are changed by the gate.

OUTPUT:

```
qc = QuantumCircuit(3)
qc.cswap(0, 1, 2)
display(qc.draw("mpl"))
print("Input q2q1q0 -> output q2q1q0")
for q2 in [0, 1]:
    for q1 in [0, 1]:
        for q0 in [0, 1]:
            test = QuantumCircuit(3)
            if q0: test.x(0)
            if q1: test.x(1)
            if q2: test.x(2)
            test.cswap(0, 1, 2)
            sv = Statevector.from_instruction(test)
            index = int(abs(sv.data).argmax())
            print(f"{q2}{q1}{q0} -> {format(index, '03b')}")
```



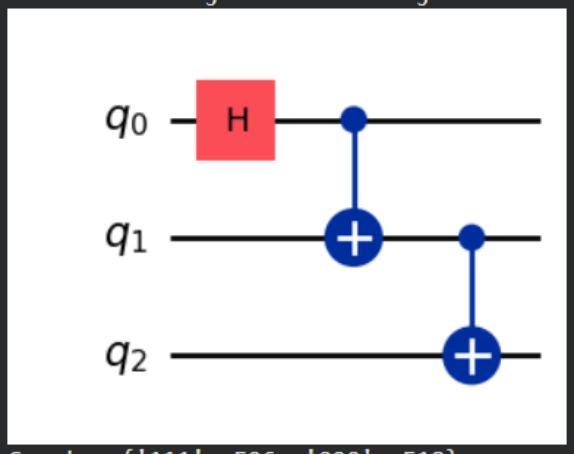

```
Input q2q1q0 -> output q2q1q0
000 -> 000
001 -> 001
010 -> 010
011 -> 101
100 -> 100
101 -> 011
110 -> 110
111 -> 111
```

Program 8: Prepare the three-qubit GHZ state using one Hadamard gate and two CNOT gates. Display the statevector and the histogram for 1024 shots, and state which outcomes never appear.

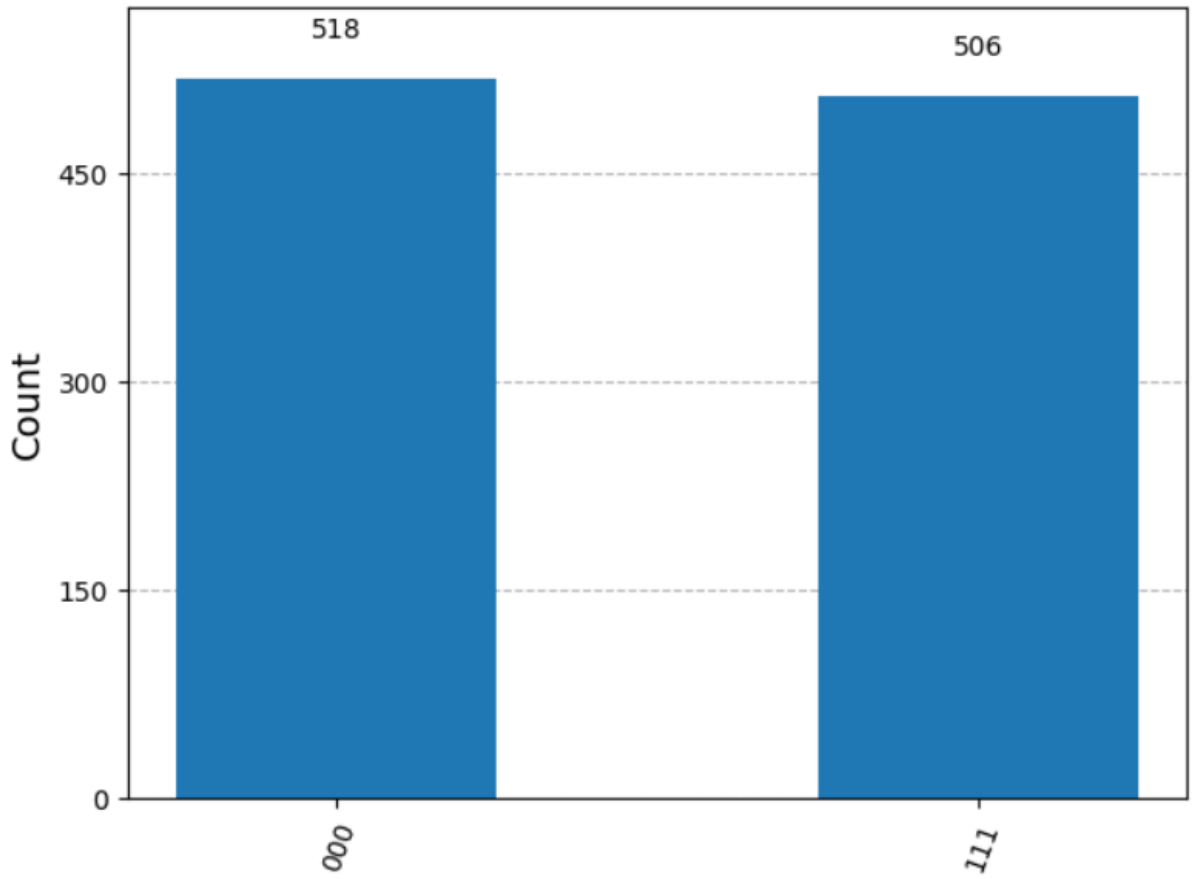
OUTPUT:

```
qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
print("GHZ statevector:", Statevector.from_instruction(qc).data)
display(qc.draw("mpl"))
measured = qc.copy()
measured.measure_all()
counts = sim.run(transpile(measured, sim), shots=SHOTS).result().get_counts()
print("Counts:", counts)
print("Outcomes that never appear:", sorted(set(format(i, "03b") for i in range(8)) - set(counts)))
display(plot_histogram(counts))
```

```
GHZ statevector: [0.70710678+0.j 0.        +0.j 0.        +0.j 0.        +0.j
 0.        +0.j 0.        +0.j 0.        +0.j 0.70710678+0.j]
```



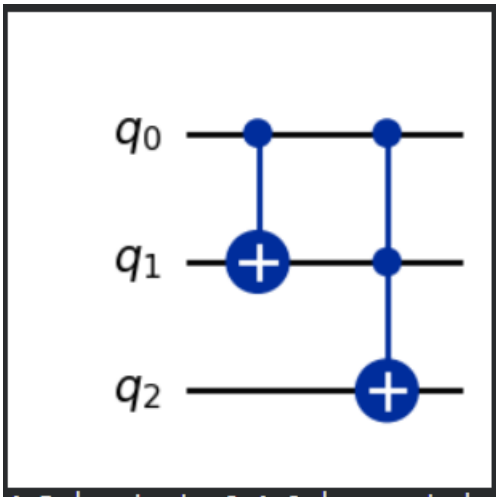

```
Counts: {'111': 506, '000': 518}
Outcomes that never appear: ['001', '010', '011', '100', '101', '110']
```



Program 9: Build a reversible half adder on three qubits using one Toffoli gate and one CNOT gate, with the third qubit starting at 0 and holding the carry. Verify the sum and carry for all four inputs.

OUTPUT:

```
qc = QuantumCircuit(3)
qc.cx(0, 1)
qc.ccx(0, 1, 2)
display(qc.draw("mpl"))
print("A B | output q2q1q0 | expected sum carry")
for a in [0, 1]:
    for b in [0, 1]:
        test = QuantumCircuit(3)
        if a: test.x(0)
        if b: test.x(1)
        test.cx(0, 1)
        test.ccx(0, 1, 2)
        sv = Statevector.from_instruction(test)
        index = int(abs(sv.data).argmax())
        bits = format(index, "03b")
        print(f"{a} {b} | {bits} | sum={a ^ b}, carry={a & b}")
```




```
A B | output q2q1q0 | expected sum carry
0 0 | 000 | sum=0, carry=0
0 1 | 010 | sum=1, carry=0
1 0 | 111 | sum=1, carry=0
1 1 | 001 | sum=0, carry=1
```

Program 10: Decompose the Toffoli gate into single-qubit and CNOT gates using `transpile` with the basis gates `h`, `t`, `tdg` and `cx`. Report the number of CNOT gates and the circuit depth, and confirm with `Operator.equiv` that the decomposed circuit is the same gate.

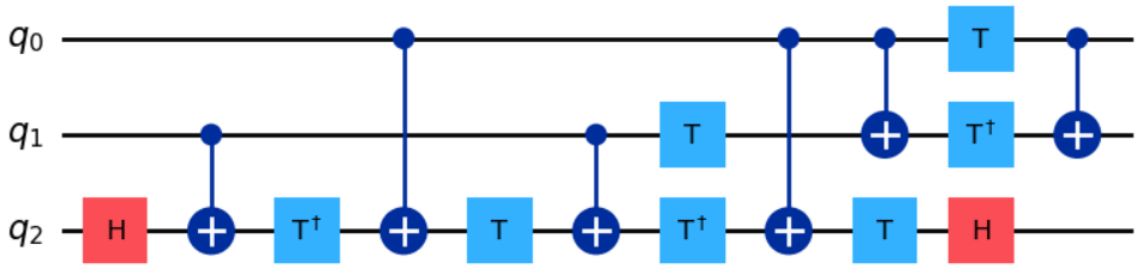
OUTPUT:

```
original = QuantumCircuit(3)
original.ccx(0, 1, 2)

decomposed = transpile(
    original,
    basis_gates=["h", "t", "tdg", "cx"],
    optimization_level=0
)

display(original.draw("mpl"))
display(decomposed.draw("mpl"))
print("Decomposed circuit:")
print(decomposed)
print("CNOT count:", decomposed.count_ops().get("cx", 0))
print("Circuit depth:", decomposed.depth())
print("Equivalent to original Toffoli:",
      Operator(original).equiv(Operator(decomposed)))
```





```
Decomposed circuit:
q_0: ──────────────■──────────────────────■──────■──────T──■──
                   │                      │      │      │  
q_1: ──────■───────┼──────────T───────────┼──────X──────Tdg─X──
           │       │          │           │                 
q_2: ──H───X───Tdg─X──T───X──Tdg─X──T──H──┼─────────────────
                          │              │                 
CNOT count: 6
Circuit depth: 11
Equivalent to original Toffoli: True
```

VIVA Q&A:

1. What is a quantum gate, and why must its matrix be unitary?

A quantum gate is an operation that changes the state of a qubit. Its matrix must be unitary to preserve the total probability of the quantum state and ensure the operation is reversible.

2. What does the Hadamard gate do to the states |0> and |1>?

The Hadamard gate creates an equal superposition of the two basis states.

$H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$

$H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$

3. State the truth table of the CNOT gate. Which qubit is the control and which is the target?

CNOT flips the target qubit only when the control qubit is 1.

Control	Target (input)	Target (output)
0	0	0
0	1	1
1	0	1
1	1	0

The control determines whether the operation occurs; the target is the qubit that may flip.

4. How is the CZ gate different from the CNOT gate?

CNOT flips the target qubit when the control qubit is |1>. CZ applies a phase change of -1 to the |11> state and leaves the other basis states unchanged.

5. What is entanglement? How is a Bell state prepared?

Entanglement is a quantum correlation in which qubits cannot be described independently. A Bell state can be prepared by applying a Hadamard gate to one qubit, followed by a CNOT gate.

For example:

$|\Phi+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$

6. What does the Toffoli gate do, and why can it reproduce classical logic circuits?

The Toffoli gate is a three-qubit controlled-controlled-NOT gate. It flips the target only when both control qubits are 1. It can implement reversible classical

logic, including AND-based operations, and is useful for constructing reversible logic circuits.

7. What does the Fredkin gate do?

The Fredkin gate is a controlled-SWAP gate. It swaps two target qubits only when the control qubit is 1.

8. How is the GHZ state different from a Bell state?

A Bell state is an entangled state of two qubits, whereas a GHZ state is an entangled state of three or more qubits.

For three qubits:

$|GHZ\rangle = (|000\rangle + |111\rangle)/\sqrt{2}$

9. In Qiskit, which qubit is printed as the rightmost bit of a result?

Qubit 0 is printed as the rightmost bit. For example, applying an X gate to qubit 0 in a two-qubit circuit produces the bitstring `01`.

10. Why must a three-qubit gate be decomposed before it runs on quantum hardware?

Quantum hardware generally supports specific one-qubit and two-qubit operations. Therefore, a three-qubit gate such as Toffoli is decomposed into supported gates before execution. A standard Toffoli decomposition uses six CNOT gates.

RESULT:

Thus, the single-qubit, two-qubit, and three-qubit quantum gates were implemented and verified using Qiskit. The truth tables and gate matrices were examined, and the Bell and GHZ entangled states were successfully prepared and simulated.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	1	4	2	1	1	1	10
Marks awarded							

Faculty signature with date: